

# Plan de Base de Datos y Despliegue: PlusBienes

Tech Lead & Senior Dev Architecture Spec

## 1. Estrategia de Base de Datos

Para la persistencia de datos en el CRM y el control de automatizaciones, utilizaremos **PostgreSQL (vía Supabase)** por su compatibilidad nativa con TypeScript y su capacidad de escalar.

### Esquema de Tablas (SQL)

```
-- Tabla de Propiedades (Core del CRM)
CREATE TABLE properties (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  title TEXT NOT NULL,
  description TEXT,
  price NUMERIC(15, 2),
  currency TEXT DEFAULT 'USD',
  location_city TEXT,
  status TEXT DEFAULT 'active', -- ['active', 'sold', 'archived']
  last_valuation_update TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Tabla de Auditoría de Tareas (Cron Jobs)
CREATE TABLE cron_job_logs (
  id BIGSERIAL PRIMARY KEY,
  job_name TEXT NOT NULL,
  execution_date TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  status TEXT, -- ['success', 'failed']
  records_processed INTEGER DEFAULT 0,
  error_details TEXT
);
```

## 2. Implementación de Procesamiento (Back-end)

Dado que el sistema actual se basa en archivos estáticos, debemos activar **Nitro (Nuxt Engine)** para manejar el procesamiento en el servidor en Hostinger.

### Configuración de API Cron (Server Route)

Ubicación: `server/api/cron/sync-prices.ts`

```
export default defineEventHandler(async (event) => {
  const config = useRuntimeConfig();
  const auth = getHeader(event, 'Authorization');

  // Validación de seguridad (Secret Token)
  if (auth !== `Bearer ${config.cronSecret}`) {
    throw createError({ statusCode: 401, message: 'Unauthorized' });
  }

  // Lógica de procesamiento de base de datos aquí...
  return { status: 'processed', timestamp: new Date().toISOString() };
});
```

### 3. Plan de Despliegue en Hostinger

Para habilitar los Cron Jobs, el despliegue debe pasar de `static` a **Node.js (SSR/Hybrid)**.

| Fase       | Acción Técnica                                                         | Herramientas              |
|------------|------------------------------------------------------------------------|---------------------------|
| Build      | Ejecutar <code>npm run build</code> para generar el bundle de Node.js. | Nuxt 4 / Nitro            |
| Transfer   | Subir el contenido de la carpeta <code>.output/</code> al servidor.    | SFTP / SSH                |
| Runtime    | Gestionar el proceso del servidor para que no muera.                   | PM2 (Process Manager)     |
| Scheduling | Configurar la llamada recurrente al endpoint de la API.                | Crontab / Hostinger Panel |

### 4. Configuración del Cron Job (Crontab)

Para activar el procesamiento automático, se debe añadir la siguiente instrucción en el gestor de tareas del servidor:

```
# Ejecutar cada día a las 00:00 (Medianoche)
0 0 * * * curl -X GET https://plusbienes.com/api/cron/sync-prices -H "Authorization: Bearer TU_TOKEN"
```

### 5. Reglas de Oro para el Desarrollo

- **Type Safety:** Generar los tipos de Supabase automáticamente con el CLI para evitar errores de esquema en el front-end.
- **Mobile First:** Verificar que las tablas del CRM sean `scrollable` en dispositivos móviles mediante Tailwind.

- **Hybrid Rendering:** Usar `routeRules` en `nuxt.config.ts` para mantener la landing estática (SSG) y el panel administrativo dinámico (SSR).